Introduction

Open sixteen cores of usage graph, run a CPU-heavy Python script, and watch fifteen of them stay flat. That's not a fluke, and it's not your laptop being slow — it's the Global Interpreter Lock, one of the most consequential design decisions in the Python language, and one that trips up engineers at every level.

Interviewers bring this up constantly, not because they want you to recite trivia about CPython internals, but because it reveals whether you actually understand what your code is doing at runtime versus what you assume it's doing. A candidate who reaches for `threading` to speed up a number-crunching loop without knowing about the GIL will confidently ship code that does nothing useful — and in production, that mistake burns real compute budget on a machine that was never actually parallelizing anything.

Learning this properly means you stop treating "concurrency" as one tool and start treating it as a question: *is this task waiting, or is this task working?* Get that distinction right, and thread vs. process stops being a coin flip.

Problem Overview

Python ships with a global lock inside the interpreter. Only one thread can hold this lock and execute Python bytecode at any given moment — no matter how many CPU cores the machine has. If you spin up ten threads on a sixteen-core machine, you don't get ten threads running in parallel. You get ten threads taking turns on one core, with the other fifteen doing nothing for your Python code specifically.

The lock exists because CPython's memory management (reference counting, primarily) isn't thread-safe by default. Rather than making every object access thread-safe individually — which is slow and complex — CPython takes the simpler route: only one thread touches Python objects at a time. That's the trade-off. Simplicity and speed for single-threaded code, at the cost of true parallelism for multi-threaded CPU work.

The lock isn't held forever, though. It gets released whenever a thread stops needing the CPU — waiting on a network response, a disk read, a timer, a database query. That's the detail that determines whether threading helps you or wastes your time.

Example

Scenario A — I/O-bound work: four HTTP requests, each taking about one second to get a response.

- Run them one after another: ~4 seconds total. Each request pays its own wait, and the waits stack up.
- Run them in four threads: ~1 second total. While thread 1 is sitting idle waiting on the network, the GIL is free, so thread 2 can run. All four waits overlap.

Scenario B — CPU-bound work: four functions, each doing fifty million additions, no waiting involved.

- Run them in four threads: time is nearly identical to running one thread alone. There's no idle moment where the GIL gets handed off — each thread is always busy, so one thread just monopolizes the lock while the others queue behind it.
- Run them in four separate processes: time drops to roughly a quarter — a real 4x speedup, because each process has its own interpreter and its own GIL, so the cores genuinely run in parallel.

The output tells the whole story: threading collapses wait time, multiprocessing multiplies compute throughput. They are not interchangeable tools for "make it faster."

Intuition

Here's how an experienced engineer reasons through this, before writing a line of code.

Start with one question: **when this task is running, is the CPU actually doing something, or is it sitting around waiting for something else to finish?**

Network calls, disk reads, database queries, and file I/O all have long stretches where the CPU is idle — the request is out on the wire, or the disk head is seeking. During that idle window, a single CPU could easily juggle other work. Threads exploit exactly this: while thread A waits for a response, thread B — or C, or D — gets the CPU. The GIL isn't a problem here because it's not being contested; nobody wants the CPU while they're waiting.

Now flip it. If a task is fifty million additions in a tight loop, there is no idle window. The CPU is pegged the entire time. Handing the GIL to another thread just means that thread also wants the CPU, immediately, with nothing to overlap. You've added the overhead of context-switching threads without gaining anything — you're still fundamentally limited to one core's worth of work, dressed up as four threads.

The fix, once you see this, is obvious: stop trying to share one interpreter's lock across CPU-bound work, and instead give each unit of work its *own* interpreter. That's what a process is. Four processes means four independent Python interpreters, four independent GILs, and — finally — four cores actually computing at once.

Brute Force Solution

Idea: Use `threading` for everything, regardless of what the work looks like, because "concurrency" sounds like it should always help.

Algorithm: 1. Spin up a thread per unit of work. 2. Start them all. 3. Join them all. 4. Assume you've parallelized the work.

Advantages: Simple to write, low memory overhead, works genuinely well for I/O-bound work.

Disadvantages: Silently fails to speed up CPU-bound work. Worse, it can look like it's "working" (the code runs, produces correct results) while delivering zero performance benefit — the kind of bug that only shows up when someone profiles production and asks why four threads take exactly as long as one.

Time complexity: For CPU-bound work, still O(total work) on a single core — no parallel reduction, despite N threads. **Space complexity:** O(N) thread stacks, each with its own overhead, for a benefit you never actually collect.

Optimal Solution

Match the tool to the bottleneck:

Work is…	Bottleneck	Use	Why
Waiting on network/disk/DB	I/O wait	`threading`	GIL releases during waits; waits overlap
Crunching numbers	CPU cycles	`multiprocessing`	Each process gets its own interpreter and GIL — true parallel execution

For CPU-bound work specifically, the mental model is: **stop sharing an interpreter.** A `multiprocessing.Pool` (or `ProcessPoolExecutor`) forks separate OS processes, each running its own instance of the Python interpreter with its own memory space and its own GIL. There's no lock contention between them because there's nothing shared to contend over — which is also the catch. Processes can't reach into each other's variables the way threads can. Any data crossing the process boundary — arguments in, results out — has to be **pickled** (serialized) and shipped across, which costs

time and memory, and which fails outright for anything unpicklable: open file handles, database connections, lambdas, closures over local state.

That's why multiprocessing isn't a free upgrade over threading — it's a different trade-off, worth paying only when the CPU-bound work is heavy enough that the 4x (or Nx) speedup outweighs the process-spawning and serialization overhead. For a small, quick computation, four fresh interpreters and pickled arguments can cost more than they save.

`concurrent.futures` gives both models the same interface, so the actual code change between them is often one line:

```
# I/O-bound
with ThreadPoolExecutor(max_workers=4) as ex:
    results = list(ex.map(fetch_url, urls))

# CPU-bound
with ProcessPoolExecutor(max_workers=4) as ex:
    results = list(ex.map(count_to_fifty_million, workloads))
```

But "same interface" doesn't mean "same guarantees." Swap `ThreadPoolExecutor` for `ProcessPoolExecutor` and anything unpicklable passed into the function will blow up or silently misbehave. The choice isn't just "waiting vs. crunching" — it's also "can what I'm passing survive being pickled."

Python Code

```
import time
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor


def fetch_url(url: str) -> str:
    """Simulate a slow network call."""
    time.sleep(1)
    return f"fetched {url}"


def count_to_fifty_million(_: int) -> int:
    """Simulate CPU-bound work: no waiting, pure computation."""
    total = 0
    for i in range(50_000_000):
        total += i
    return total


def run_io_bound(urls: list[str]) -> list[str]:
```

```
    with ThreadPoolExecutor(max_workers=len(urls)) as executor:
        return list(executor.map(fetch_url, urls))


def run_cpu_bound(n_workers: int) -> list[int]:
    with ProcessPoolExecutor(max_workers=n_workers) as executor:
        return list(executor.map(count_to_fifty_million, range(n_workers)))


if __name__ == "__main__":
    urls = ["url_1", "url_2", "url_3", "url_4"]

    start = time.perf_counter()
    run_io_bound(urls)
    print(f"I/O-bound (threads): {time.perf_counter() - start:.2f}s")

    start = time.perf_counter()
    run_cpu_bound(4)
    print(f"CPU-bound (processes): {time.perf_counter() - start:.2f}s")
```

Code Walkthrough

- `fetch_url` sleeps for a second — a stand-in for any real I/O wait (HTTP call, DB query, disk read). The sleep is what releases the GIL, letting other threads run.
- `count_to_fifty_million` never waits on anything external; it's a pure CPU loop, deliberately heavy enough to make the GIL's effect visible.
- `run_io_bound` uses `ThreadPoolExecutor` sized to the number of URLs, so all four requests are in flight at once, overlapping their wait time.
- `run_cpu_bound` uses `ProcessPoolExecutor`, so each unit of work gets its own OS process and interpreter — genuine parallel execution across cores.
- `executor.map` in both cases hides the manual thread/process start-join bookkeeping — you get results back in order, without managing lifecycle by hand.
- The `if __name__ == "__main__":` guard matters specifically for multiprocessing on Windows and macOS (spawn start method): without it, child processes re-import the module and can recursively spawn more children.

Dry Run

Take `run_io_bound(["url_1", "url_2", "url_3", "url_4"])`:

1. Four threads start almost simultaneously, each calling `fetch_url` with its own URL.

2. Each thread hits `time.sleep(1)`. The GIL is released the instant a thread enters the sleep, since sleeping isn't executing Python bytecode.
3. With the GIL free, another thread's `fetch_url` call gets scheduled and also hits its sleep.
4. All four sleeps proceed concurrently in wall-clock time, even though only one thread could theoretically hold the GIL doing Python work at once — sleeping isn't "Python work."
5. After ~1 second, all four wake up, threads finish, `executor.map` collects results in order: `["fetched url_1", "fetched url_2", "fetched url_3", "fetched url_4"]`.
6. Total elapsed time: ~1 second, not ~4.

Now `run_cpu_bound(4)`:

1. Four OS processes are spawned, each with its own interpreter instance.
2. Each process runs `count_to_fifty_million` independently — no shared GIL, no waiting on each other.
3. All four loops execute on separate cores simultaneously.
4. Results return to the parent process (pickled across the process boundary), collected in order.
5. Total elapsed time: roughly a quarter of running all four sequentially in one process.

Complexity Analysis

Threading (I/O-bound): - Time: O(max wait), not O(sum of waits) — the dominant cost is the longest single wait, since waits overlap. Correct because idle time doesn't compete for the GIL. - Space: O(N) thread stacks, cheap relative to processes.

Multiprocessing (CPU-bound): - Time: O(total work / N workers), bounded below by core count and serialization overhead. Correct because each process genuinely computes in parallel, unlike threads sharing one GIL. - Space: O(N × interpreter overhead) — each process duplicates interpreter startup cost and memory, which is why multiprocessing is not "free" the way threading is.

Alternative Solutions

`asyncio` is worth mentioning as a third option for I/O-bound work: instead of OS threads, it uses a single-threaded event loop with cooperative coroutines. It scales to far more concurrent I/O operations than threads can (thousands of open connections vs. hundreds of threads) with lower memory overhead, but it requires `async` / `await`-aware libraries throughout your call stack — you can't just drop a blocking `requests.get()` into it and expect the same overlap benefit you get from threads.

Compared to `asyncio`, threading is simpler to retrofit onto existing blocking code; `asyncio` is the better long-term choice for high-concurrency I/O services built from scratch.

Edge Cases

- **Zero tasks:** `executor.map` over an empty iterable returns immediately with no work — handle a truly empty batch before spinning up a pool at all, since pool creation still has overhead.
- **A single task:** threading/multiprocessing overhead can exceed the benefit; for one task, just call the function directly.
- **A task that raises an exception:** with `executor.map`, the exception surfaces when you iterate the results, not when the task runs — don't assume silence means success.
- **Unpicklable arguments in `ProcessPoolExecutor`:** passing a lambda, open file handle, or DB connection raises a `PicklingError` (or hangs, depending on platform) — pass plain data and open resources inside the worker function instead.
- **More workers than cores for CPU-bound work:** doesn't speed things up past the core count; it just adds context-switching overhead between processes competing for the same physical cores.

Common Mistakes

1. **Reaching for `threading` by default for anything called "concurrency," without checking if the work is CPU-bound.** This is the single most common GIL mistake — the code runs, produces correct output, and delivers zero speedup.
2. **Assuming `multiprocessing` is always faster.** For light work, process-spawn and pickling overhead can make it slower than just running sequentially.
3. **Passing unpicklable objects (open files, DB connections, lambdas) into `ProcessPoolExecutor`.** Works fine in threads, breaks silently or loudly in processes.
4. **Forgetting the `if __name__ == "__main__":` guard when using multiprocessing**, causing runaway process spawning on Windows/macOS.
5. **Assuming threads share no state, when they actually share all state** — leading to race conditions on shared mutable objects that don't exist across process boundaries.
6. **Sizing a thread or process pool arbitrarily** instead of matching pool size to the actual bottleneck (I/O concurrency limits vs. CPU core count).

Interview Questions

- What is the GIL, and why does CPython have it?
- Why does threading speed up I/O-bound Python code but not CPU-bound code?
- How does multiprocessing avoid the GIL, and what does that cost?
- What happens if you pass a lambda or open file handle into a `ProcessPoolExecutor`?

- When would you choose `asyncio` over `threading` for I/O-bound work?
- Is the GIL removed in newer versions of Python? (Worth knowing: PEP 703 introduces an experimental free-threaded build without the GIL, though it's not yet the default.)

Similar Problems

- **Producer-Consumer with `queue.Queue`** — classic thread-safe coordination problem that builds on understanding what the GIL does and doesn't protect.
- **Rate-limited API fetcher** — extends the threaded I/O pattern with concurrency limits and backoff.
- **Parallel merge sort using multiprocessing** — a CPU-bound divide-and-conquer problem that's a natural fit for process pools.
- **Dining Philosophers / deadlock problems** — general concurrency reasoning that transfers across languages, not GIL-specific but tests the same "what's actually running when" intuition.

Key Takeaways

- The GIL means only one thread executes Python bytecode at a time — full stop.
- Threads help when work waits (I/O); the GIL releases during that wait.
- Threads don't help when work computes (CPU-bound); there's no idle moment to hand off.
- Multiprocessing sidesteps the GIL by giving each process its own interpreter — real parallelism, at the cost of memory and serialization.
- `concurrent.futures` unifies the interface, but the pickling requirement for processes is not optional — check what you're passing across the boundary.

Watch the Video

This lesson is easier to absorb watching the core-usage graph react in real time to threaded vs. process-based code. Catch the full walkthrough here: {LINK}

About the Series

This is Lesson 32 of *Fun with Learning Technology* — a series that takes real developer topics (Python internals, algorithms, system design, and interview-relevant engineering) and breaks them down the way a senior engineer would explain them to a teammate: no fluff, just the reasoning that makes the concept click.

Call To Action

If this saved you from shipping a "parallel" script that wasn't, drop a like on the video and subscribe on YouTube for the rest of the series. Subscribe to the Substack for the deeper written breakdowns that don't fit in a video. Got a GIL war story or a concurrency bug that cost you a day? Drop it in the comments — and share this with anyone still threading their way through a CPU-bound loop.

Quick Review

Why doesn't threading speed up CPU-bound Python code?

The GIL lets only one thread execute Python bytecode at a time, so threads serialize on CPU work.

When to reach for threading vs multiprocessing?

Threading for I/O-bound waits (network, disk); multiprocessing for CPU-bound work needing real parallelism.

Why do processes sidestep the GIL?

Each process gets its own interpreter and memory space, so GIL contention doesn't cross process boundaries.

Common bug when picking concurrency for CPU-bound tasks?

Using ThreadPoolExecutor for CPU work; threads still contend on the GIL, so it acts like single-threaded code.

Cost of multiprocessing vs threading?

Processes aren't free: extra memory, slower startup, and pickling overhead to pass data between processes.

What does concurrent.futures unify?

ThreadPoolExecutor and ProcessPoolExecutor share one Future-based API, so you swap executor class by workload type.

Python Lesson 32: Concurrency: Threads and Processes (threading, multiprocessing, the GIL, and concurrent.futures) — free Python cheat sheet